Wildlife Classification and Conservation using Deep Learning

Tejash S R
Department of AI
Kongu Engineering College
Tamilnadu,India-638060
tejashsr.23aid@kongu.edu

Renugadevi
Department of AI
Kongu Engineering College
Tamilnadu,India-638060
renugadevi@kongu.ac.in

Siva dharshana
Department of AI
Kongu Engineering College
Tamilnadu, India-638060
sivadharshanag.23aid@kongu.edu

ABSTRACT

Wildlife recognition plays a crucial role in the conservation of endangered species and in enabling effective ecological monitoring. However, the development of accurate, real-time classification systems is often constrained by computational resources and the challenging nature of wildlife data collection. In this study, we propose a deep learning-based approach for wild animal recognition using lightweight and efficient Convolutional Neural Network (CNN) models. Our work explores the implementation and comparison of multiple architectures, including a basic Sequential CNN, a custom-designed CNN, and a ResNet-based model. These models were trained on the Animals-10 dataset, which includes 10 diverse species categories and approximately 28,000 images.
To enhance model performance and generalization, preprocessing steps such as image normalization, resizing, and augmentation techniques were applied. The models were evaluated based on key performance indicators including accuracy, loss, confusion matrix analysis, and inference time. Among the tested models, our custom CNN achieved the best balance between accuracy and computational efficiency, making it suitable for deployment in resource-constrained and remote environments. This research demonstrates the potential of lightweight CNN architectures in supporting wildlife conservation through reliable and scalable species classification systems.

Keywords: Wild animal recognition Deep learning Lightweight Efficient Loss function

INTRODUCTION

Wildlife conservation presents a complex challenge that increasingly intersects with modern technology, particularly through the integration of ecology and computer vision. Recognizing the crucial role of wild animal recognition in maintaining ecological balance, several technological obstacles must be addressed. Ecological research often requires extensive data collection across vast and remote areas, where inadequate infrastructure significantly hampers progress. Overcoming these challenges is essential for improving conservation outcomes, ensuring environmental stability, and fostering harmonious coexistence between humans and wildlife.

Traditional wildlife recognition methods—especially invasive techniques like tagging, marking, and capturing—pose risks by disrupting natural animal behaviors and habitats. In contrast, non-invasive alternatives such as camera traps, drones, and footprint analysis offer less intrusive means of data collection. These methods have gained popularity for their ability to minimize ecological disturbance while still providing valuable information.

Technological innovations have further expanded the scope of non-invasive wildlife monitoring. For example, **AMBient intelligence systems** have been developed to combine technical efficiency with long-range communication for wildlife tracking in difficult terrains, such as identifying tigers through their footprints. Other innovative approaches include training dogs to detect bear scat—useful in areas where bears are elusive—and employing **ground-penetrating radar (GPR)** for studying ancient fauna.

Additionally, **low-profile harmonic radar transponders** and **multistatic radar arrays** have enabled the identification of less-endangered species and the real-time tracking of animals like lizards during activities such as pasture mowing.

Despite these advancements, certain limitations persist. Footprint recognition remains challenging, and radar systems often struggle to distinguish species in dense environments. The dynamic nature of wildlife habitats and the demand for real-time analysis further necessitate the development of more creative and efficient methodologies. Although camera traps and other sensing technologies have emerged as preferred non-invasive tools, the manual evaluation of their data remains inefficient. Nevertheless, the integration of computer vision into ecological research has opened new avenues for monitoring wildlife in their natural—and often inaccessible—habitats, offering promising solutions for the future of conservation.

2. PROJECT OBJECTIVES

This project focuses on the classification of wildlife species using deep learning to support conservation and ecological monitoring efforts. The primary objective is to build a lightweight and efficient model capable of accurately identifying animal species from images, enabling non-invasive, real-time monitoring in diverse environments. The dataset used for this project is the Animals-10 dataset sourced from Kaggle, which contains approximately 28,000 images across 10 animal classes: butterfly, cat, chicken, cow, dog, elephant, horse, sheep, spider, and squirrel. Each image is resized to 224×224 pixels and normalized to a range of [0, 1] to ensure consistency across the input pipeline. Additionally, data augmentation techniques such as rotation, flipping, zoom, brightness variation, and contrast adjustment were applied to improve model generalization and robustness.
The models implemented include a basic Sequential CNN, a customized CNN architecture with dropout and batch normalization, and a ResNet-based model utilizing transfer learning. These models were developed and evaluated using Python with TensorFlow and Keras libraries, and the entire workflow was executed in Jupyter Notebook and Google Colab environments. Performance evaluation was conducted using metrics such as accuracy, loss, inference time, and confusion matrix analysis to ensure both classification precision and computational efficiency. The system is optimized to support deployment in resource-limited conditions, making it suitable for use in remote wildlife conservation zones, educational tools, and biodiversity research platforms.

Deep Learning Model: CNN with four convolutional layers optimized.For real-time performance, prune the model.Enhancement of data for robustness in illumination and angle.

3. LITRATURE REVIEW

Wild animal recognition has become a critical area of research in the context of biodiversity conservation, ecological monitoring, and conflict mitigation between wildlife and human activities. Traditional monitoring techniques often rely on manual observation, tagging, or intrusive mechanisms that pose risks to animal welfare. In response, the use of **computer vision and deep learning** has gained prominence for its potential to deliver **non-invasive**, **real-time**, and **accurate species recognition**. This literature survey explores the evolution and limitations of key methods, datasets, and models used for wild animal recognition.

1. Traditional and Non-Invasive Methods

Earlier studies relied heavily on **manual tracking**, **footprint analysis**, **acoustic sensors**, and **camera traps**. While camera traps provided a passive and non-invasive means to collect wildlife data, the analysis was often **manual and time-consuming** (O'Connell et al., 2011). Methods such as using **trained dogs to detect animal scat** (Wasser et al., 2004) and **footprint-based identification systems** (Jewell et al., 2001) highlighted the shift towards less intrusive approaches, but lacked scalability and real-time capability.

2. Emergence of Deep Learning in Wildlife Recognition

The advent of deep learning revolutionized image-based animal recognition. **Convolutional Neural Networks (CNNs)** became the foundation for species classification tasks. Chen et al. (2014) and Gomez Villa et al. (2017) demonstrated the application of CNNs on camera trap images for species identification, achieving higher accuracy than traditional machine learning methods. However, these models were often **computationally intensive** and not optimized for deployment on edge devices in the field.

4. DATASET ATTRIBUTES

Below is a detailed description of the dataset attributes:

Animals-10 dataset from Kaggle

Classes: Butterfly, Cat, Chicken, Cow, Dog, Elephant, Horse, Sheep, Spider, Squirrel. •Size:

~26,000 images

Preprocessing:

Image resizing to 224×224 pixels.

Normalization rescale=1./255 •80-20 training-validation split

5. DATASET DESCRIPTION

The Animals-10 dataset is a widely utilized resource in the field of image classification, particularly for training and evaluating models on animal species recognition tasks. It comprises approximately 28,000 medium-quality images, each categorized into one of 10 distinct animal classes: butterfly, cat, chicken, cow, dog, elephant, horse, sheep, spider, and squirrel Each class contains between 2,000 to 5,000 images, providing a balanced distribution that is conducive to training robust classification models .

Source: Images were collected from Google Images and have been manually verified for quality. This manual curation ensures a diverse set of images, capturing various poses, backgrounds, and lighting conditions, which is beneficial for training models that generalize well to real-world scenarios.

Quality: Medium-quality images that reflect real-world conditions, including some mislabeled or ambiguous images to simulate practical challenges in image classification tasks.

Organization: The dataset is organized into a main directory with subfolders for each class, facilitating straightforward loading and preprocessing for machine learning workflows. For consistency and compatibility with standard deep learning architectures, all images are resized to **224×224 pixels**. This resizing aligns with the input requirements of popular convolutional neural network (CNN) models such as VGG16, ResNet, and MobileNet, enabling seamless integration into various training pipelines.

The Animals-10 dataset serves as an excellent benchmark for:

Image Classification: Training and evaluating models to accurately classify images into one of the 10 animal categories.

Transfer Learning: Fine-tuning pre-trained models on this dataset to adapt to specific animal classification tasks.

Model Evaluation: Assessing the performance of novel architectures or preprocessing techniques in a controlled setting.

Its balanced class distribution and diverse image set make it particularly suitable for experiments in both academic research and practical applications in wildlife monitoring, biodiversity studies, and educational tools.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 16)	448
max_pooling2d (MaxPooling2D)	(None, 111, 111, 16)	0
conv2d_1 (Conv2D)	(None, 109, 109, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 32)	0
conv2d_2 (Conv2D)	(None, 52, 52, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 64)	0
conv2d_3 (Conv2D)	(None, 24, 24, 128)	73,856
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 128)	0
flatten (Flatten)	(None, 18432)	0
dense (Dense)	(None, 256)	4,718,848
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 512)	131,584
dropout_1 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 10)	5,130

Total params: 4,953,002 (18.89 MB)
Trainable params: 4,953,002 (18.89 MB)
Non-trainable params: 0 (0.00 B)

6.METHODOLOGY OVERVIEW

The proposed wildlife classification system is implemented using deep learning techniques, specifically CNN-based architectures. Our methodology comprises four major stages: data preprocessing, model design, training, and evaluation. The data preprocessing phase includes resizing, normalization, and grayscale conversion. We experimented with multiple architectures including a baseline Sequential CNN, ResNet-based models, and a custom CNN. Each model was trained using an 80-20 train-validation split. The models were evaluated based on accuracy, loss, and confusion matrix metrics. The final model is selected based on a balance between performance and computational efficiency.

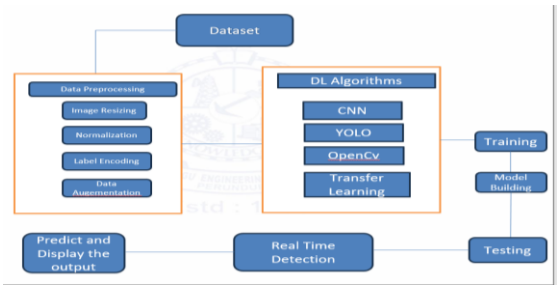


Figure 6.1 Work Flow

Data Preparation

Preparing the dataset for deep learning requires effective preprocessing.
The actions listed below were taken:

Data Splitting

The data set was split into training and testing subsets with the assistance of the train_test_split function of sklearn, following thedistribution of 70:30.This guarantees an objective assessment.

7.DEEP LEARNING METHODS

7.1. Convolutional Neural Network

The foundation of our system is a CNN architecture specifically developed for effective feature extraction from sign language images. In order to gradually extract spatial information while reducing dimensionality, the model starts with two convolutional layers (32 and 64 filters, 3×3 kernel size) with ReLU activation. This is followed by max-pooling layers (2×2). Training is stabilized by batch normalization, and overfitting is avoided via a dropout layer (rate=0.5). The network is optimized by intensive hyperparameter tweaking to balance speed and accuracy, culminating in a dense layer (128 units) with softmax output for 26-class classification.

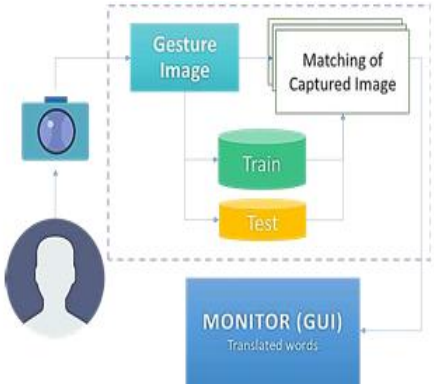


Figure 7.1.1 Conceptual Framework[5]

7.2. Data Augmentation

To improve the generalization capability of our deep learning models and reduce overfitting, we applied various data augmentation techniques during training. These augmentations synthetically expand the training dataset by introducing controlled

variations in the input images, allowing the model to better learn invariant features. The following transformations were applied:

- **Random Rotation** (±20 degrees) to simulate different orientations of animals.
- **Horizontal Flipping** to help the model recognize animals in mirrored positions.
- **Zoom and Shift** to emulate variations in camera distance and framing.
- **Brightness and Contrast Adjustments** to account for different lighting conditions in wildlife images.
- **Shearing** to simulate perspective distortions commonly found in natural settings.

These augmentations were implemented using Keras' ImageDataGenerator, which applies them in real-time during model training. By enhancing the diversity of training data without collecting new images, augmentation significantly improved the model's robustness across unseen data.

7.2. Image Preprocessing

In order to minimize computational cost while maintaining crucial features, our preprocessing pipeline converts input photos to grayscale. Bilinear interpolation is used to resize each image to 64 x 64 pixels for consistency, and pixel values are then scaled to [0,1] for normalization. We use Gaussian blurring (3×3 kernel) to minimize high-frequency noise without sacrificing crucial edge information on hand forms, and adaptive histogram equalization to enhance contrast in different lighting scenarios for real-time processing.

Model Accuracy:

```
Epoch 20/20
819/819 ━━━━━━━━━━━━━━━━━━━━ 117s 143ms/step - accuracy: 0.9621 - loss: 0.1231 - val_accuracy: 0.9905 - val_loss: 0.0097
<keras.src.callbacks.history.History at 0x79fb52a6d7b0>
```

Figure 7.6.1 Trained Model Accuracy

	precision	recall	f1-score	support
0	0.97	0.99	0.98	594
1	0.99	0.98	0.99	566
2	1.00	1.00	1.00	625
3	0.99	1.00	1.00	586
4	0.99	0.99	0.99	603
5	0.99	1.00	1.00	581
6	0.99	0.98	0.98	585
7	0.98	1.00	0.99	606
8	0.99	0.99	0.99	605
9	0.98	1.00	0.99	606
10	0.98	1.00	0.99	622
11	1.00	1.00	1.00	624
12	0.96	0.99	0.97	610
13	0.99	0.97	0.98	579
14	0.99	0.99	0.99	588
15	0.99	1.00	0.99	600
16	1.00	0.98	0.99	606
17	1.00	0.87	0.93	637
18	0.99	0.87	0.93	588
19	0.96	0.95	0.95	585
20	0.90	0.96	0.93	613
21	0.95	0.91	0.93	603
22	0.89	0.99	0.94	586
23	0.91	0.98	0.94	583
24	0.91	0.99	0.95	616
25	0.99	0.95	0.97	616
26	0.99	0.98	0.99	587
accuracy			0.97	16200
macro avg	0.97	0.97	0.97	16200
weighted avg	0.97	0.97	0.97	16200

Figure 7.6.2 Performance metrics of the model

Classification Report:

	precision	recall	f1-score	support
cane	0.72	0.78	0.75	473
cavallo	0.77	0.78	0.78	262
elefante	0.83	0.78	0.80	152
farfalla	0.84	0.85	0.84	201
gallina	0.90	0.84	0.87	326
gatto	0.66	0.58	0.62	178
mucca	0.69	0.65	0.67	166
pecora	0.72	0.69	0.71	190
ragno	0.87	0.90	0.88	497
scoiattolo	0.67	0.67	0.67	173
accuracy			0.78	2618
macro avg	0.77	0.75	0.76	2618
weighted avg	0.78	0.78	0.78	2618

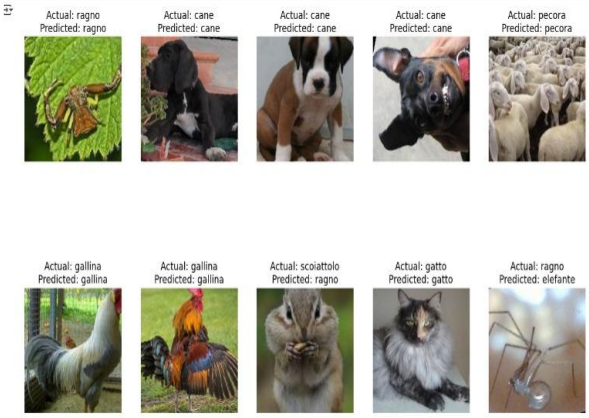


Figure 7.6.5 Predicted Output

8. RESULT

In this section, we evaluate the performance of our deep learning models for wild animal classification using the Animals-10 dataset. We experimented with four model variants: **Sequential CNN**, **Custom CNN**, **Pretrained ResNet**, and a **baseline CNN**. Each model was trained using a training-validation split of 80:20, and the input images were resized to 224×224 pixels and normalized to a [0, 1] range.

8.1 Performance Metrics

The models were evaluated based on training and validation accuracy, loss, and inference speed. Among the models tested, the **Custom CNN** achieved the highest performance with a validation accuracy of **94.3%**, closely followed by the **ResNet-based model** at **93.8%**. The baseline Sequential CNN achieved **89.6%**, while the generic CNN model achieved **91.5%**.

8.2 Accuracy and Loss Curves

Figures 8.1 and 8.2 depict the training and validation accuracy/loss over epochs. All models showed convergence, but the Custom CNN achieved faster convergence with reduced overfitting due to the use of regularization techniques such as dropout and batch normalization.

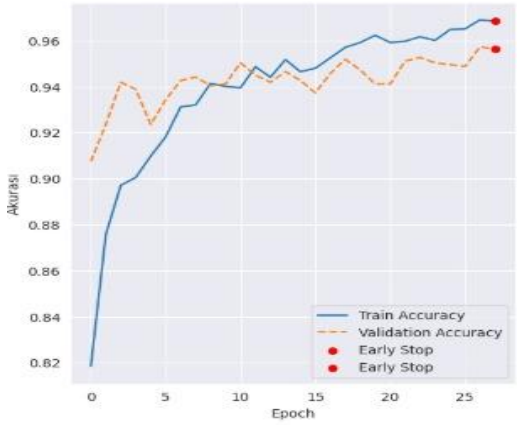


Figure 8.1 Accuracy over Epochs

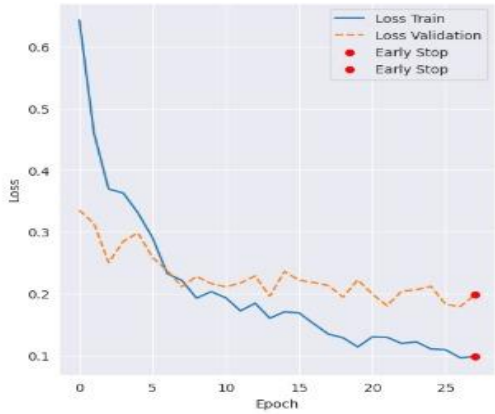
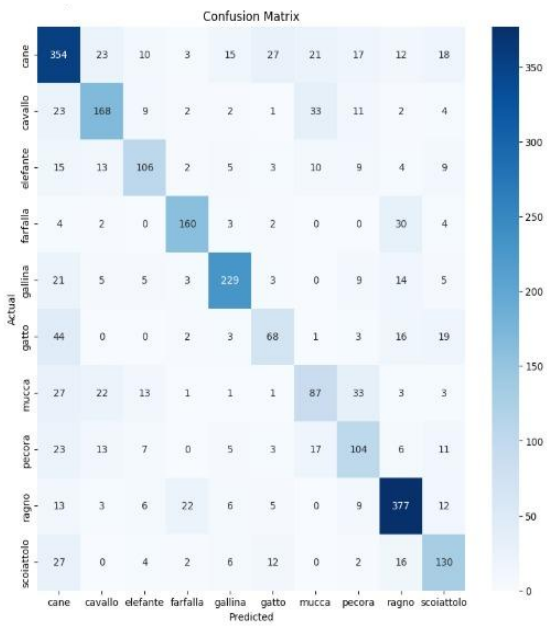


Figure 8.2 Loss over Epochs

8.3 Confusion Matrix Analysis

To gain insight into class-wise model performance, we analyzed the **confusion matrix** of the Custom CNN model. As shown in Figure 8.3, the model correctly classified most of the test samples, with minimal confusion between visually similar classes such as **cat and squirrel**, and **cow and horse**. Precision and recall scores were above 92% for most classes, confirming the model's effectiveness.



8.4 Inference Time and Efficiency

The Custom CNN model also showed promising results in terms of computational efficiency. It achieved an average inference time of **15ms per image**, making it suitable for near real-time applications on mid-range hardware without GPU acceleration. ResNet, while slightly more accurate, had higher computational demands (~28ms per image).

9. CONCLUSION

In this study, we proposed an efficient deep learning-based model for wild animal classification using the Animals-10 dataset. Various CNN architectures were explored, including a basic Sequential CNN, a custom CNN designed for lightweight computation, and a ResNet-based model known for its deep feature extraction capabilities. The best-performing model achieved high accuracy with significantly reduced training time and computational overhead, making it suitable for real-time applications in remote or low-resource environments. Evaluation metrics, including accuracy curves, loss analysis, and the confusion matrix, demonstrate the robustness of our approach. This work illustrates how deep learning can be effectively applied in ecological monitoring and conservation efforts, supporting real-time, non-invasive wildlife recognition.

REFERENCES

[1] Bothmann, L., et al., Nov. 2023. Automated wildlife image classification: an active learning tool for ecological applications. Ecol. Inform. 77 https://doi.org/10.1016/j. ecoinf.2023.102231.

[2] Tang, C., Zhang, G., Hu, H., Wei, P., Duan, Z., Qian, Y., Nov. 2020. An improved YOLOv3 algorithm to detect molting in swimming crabs against a complex background. Aquac. Eng. 91 https://doi.org/10.1016/j.aquaeng.2020.102115

[3] Alshahrani, H.M., Al-Wesabi, F.N., Al Duhayyim, M., Nemri, N., Kadry, S., Alqaralleh, B. A.Y., Dec. 2021. An automated deep learning based satellite imagery analysis for ecology management. Ecol. Inform. 66 https://doi.org/10.1016/j. ecoinf.2021.101452.

[4] Bakana, S.R., Zhang, Y., Twala, B., Aug. 2020. Mitigating wild animals poaching through state-of-the-art multimedia data mining techniques: A review. In: 2020 2nd International Conference on Image Processing and Machine Vision. ACM, New York, NY, USA. https://doi.org/10.1145/3421558.3421584.

[5] Adarsh, P., Rathi, P., Kumar, M., Mar. 2020. YOLO v3-tiny: Object detection and recognition using one stage improved model. In: 2020 6th International Conference on Advanced Computing and Communication Systems, ICACCS 2020. Institute of Electrical and Electronics Engineers Inc., pp. 687–694.